

OpsWatch CLI - Lernchat

Gespeichert am: 29.09.2026

Projekt: OpsWatch CLI

Schwerpunkt: CPU-Monitoring, Clean Code und Tests mit pytest

Bisher behandelter Lernstand

- Python-Funktion main() und if __name__ == "__main__"
- Variablen, Datentypen und Type Hints
- Bedingungen mit if, elif und else
- Funktionen und Rückgabewerte
- Modularisierung mit cpu.py, deployment.py und status.py
- Abhängigkeiten mit uv
- Codeprüfung und Formatierung mit Ruff
- Git-Branches, git status und git diff
- Echte CPU-Daten mit psutil
- CPU-Grenzwerte und Statusstufen
- Grundlagen automatisierter Tests mit pytest

Aktueller Projektstand

OpsWatch misst die CPU-Auslastung über `cpu_load()`. Die Funktion `evaluate_cpu_usage()` soll einen kontrollierten Messwert bewerten und eine von drei Meldungen zurückgeben:

- bis einschließlich 60,0 Prozent: CPU load: Normal
- über 60,0 Prozent: CPU load: Increased
- über 80,0 Prozent: Warning: High CPU load!

Empfohlener Stand von `main.py`

```
"""OpsWatch CLI: Command-line tool for system monitoring."""

from cpu import cpu_load
from deployment import describe_deployment
from status import evaluate_status

CPU_WARNING_THRESHOLD = 80.0
CPU_INCREASED_THRESHOLD = 60.0

def evaluate_cpu_usage(cpu_workload: float) -> str:
    """Return a status message for the given CPU workload."""
    if cpu_workload > CPU_WARNING_THRESHOLD:
        return "Warning: High CPU load!"
    if cpu_workload > CPU_INCREASED_THRESHOLD:
        return "CPU load: Increased"
    return "CPU load: Normal"

def main() -> None:
    """Run the main program."""
    version = "0.1.0"
    status_code = 0

    deployment_status = describe_deployment(successful=True)
    status = evaluate_status(status_code)
```

```

cpu_workload = cpu_load()
cpu_msg = evaluate_cpu_usage(cpu_workload)

print("OpsWatch CLI running!")
print(f"Deployment: {deployment_status}")
print(f"Version: {version}")
print(f"Status: {status}")
print(f"CPU load: {cpu_workload:.1f} %")
print(cpu_msg)

if __name__ == "__main__":
    main()

```

Wichtige Erkenntnisse aus dem Code Review

Gültigkeitsbereich von Variablen

Lokale Variablen aus `main()` sind nicht automatisch in anderen Funktionen verfügbar. Der CPU-Wert wird deshalb als Parameter an `evaluate_cpu_usage()` übergeben. Feste Grenzwerte können als Konstanten auf Modulebene definiert werden.

Reihenfolge der Bedingungen

Der höchste Grenzwert muss zuerst geprüft werden. Ein Wert von 90,0 liegt sowohl über 60,0 als auch über 80,0. Würde zuerst `> 60.0` geprüft und direkt ein Ergebnis zurückgegeben, könnte die Warnstufe nicht mehr erreicht werden.

Grenzfälle

Bei einem Vergleich mit `>` gilt:

- 60.0 -> Normal
- 60.1 -> Increased
- 80.0 -> Increased
- 80.1 -> Warning

Klare Namen

- `cpu_load()` misst die Auslastung.
- `cpu_workload` enthält den gemessenen Zahlenwert.
- `evaluate_cpu_usage()` bewertet den Wert.

pytest einrichten

pytest wird als Entwicklungsabhängigkeit installiert:

```
uv add --dev pytest
```

Empfohlene Projektstruktur:

```

opswatch/
├── main.py
├── cpu.py
├── deployment.py
├── status.py
├── pyproject.toml
├── uv.lock
├── tests/
│   └── test_main.py

```

Erster pytest-Test

In tests/test_main.py:

```
from main import evaluate_cpu_usage

def test_evaluate_cpu_usage_returns_normal() -> None:
    result = evaluate_cpu_usage(50.0)

    assert result == "CPU load: Normal"
```

assert prüft, ob das tatsächliche Ergebnis dem erwarteten Ergebnis entspricht. pytest erkennt Testfunktionen am Präfix test_.

Empfohlene Grenzwerttests

```
from main import evaluate_cpu_usage

def test_evaluate_cpu_usage_returns_normal() -> None:
    assert evaluate_cpu_usage(50.0) == "CPU load: Normal"

def test_evaluate_cpu_usage_at_increased_boundary() -> None:
    assert evaluate_cpu_usage(60.0) == "CPU load: Normal"

def test_evaluate_cpu_usage_returns_increased() -> None:
    assert evaluate_cpu_usage(60.1) == "CPU load: Increased"

def test_evaluate_cpu_usage_at_warning_boundary() -> None:
    assert evaluate_cpu_usage(80.0) == "CPU load: Increased"

def test_evaluate_cpu_usage_returns_warning() -> None:
    assert evaluate_cpu_usage(80.1) == "Warning: High CPU load!"
```

Befehle zum Prüfen

```
uv run pytest -v
uv run ruff check .
uv run ruff format --check .
git status
git diff
```

Git-Schritt

Wenn alle Tests erfolgreich sind:

```
git add main.py tests/test_main.py pyproject.toml uv.lock
git commit -m "test: add CPU usage evaluation tests"
```

Nächster Lernschritt

Die fünf ähnlichen Testfälle können anschließend mit `@pytest.mark.parametrize` zusammengefasst werden. Danach bietet sich an, die CPU-Bewertung aus main.py in ein fachlich passendes Modul zu verschieben.

Sicherheitsnotiz

Keine vertraulichen System-, Unternehmens- oder Zugangsdaten in ein öffentliches Repository committen.